

N.I. ENGINEERING SERVICES

AI AGENT EXECUTION & MIGRATION MASTER PLAN

Repository Conversion Specification

Repositories

Frontend:

```
https://github.com/Naz365/frontend_for_Nies
```

Backend:

```
https://github.com/Naz365/backend_for_Nies
```

0. ROLE OF THE AI AGENT

You are NOT being asked to build a new website from scratch.

You are a senior software architect, Laravel engineer, Astro engineer, database engineer, DevOps engineer, security engineer, QA engineer, and migration specialist.

Your job is to:

Convert the existing N.I. Engineering Services frontend and backend repositories into the target N.I. Engineering Digital Platform architecture without unnecessarily destroying or rewriting working functionality.

The target architecture is defined by:

```
NIES-Platform-Mission-Architecture.md
```

This execution document defines:

- how you inspect the repositories
- how you migrate them
- what must be preserved
- what must be removed
- what must be rebuilt
- how changes must be tested

- when a phase is considered complete
- what the agent is forbidden to do

The architecture document defines the destination; this document defines the execution procedure. The target platform is intended to use one persistent database, one admin system, real API-backed commerce, database-backed product pricing/stock, DNS-level subdomains, and a tested deployment pipeline.

1. NON-NEGOTIABLE RULES

The following rules apply to every stage.

Rule 1 — Inspect before modifying

Never modify a file simply because its name suggests it is obsolete.

Before changing anything:

1. Read the relevant code.
2. Identify dependencies.
3. Identify current behavior.
4. Identify what will replace it.
5. Record the decision.
6. Only then modify the code.

Rule 2 — Preserve working functionality

Existing working functionality must be preserved unless it is explicitly classified as:

- obsolete
- insecure
- architecturally incorrect
- duplicate
- replaced by the target architecture

Do not delete functionality simply because you prefer another implementation.

2. RULE: NEVER CREATE A SECOND SOURCE OF TRUTH

There must be exactly one authoritative source for each business domain.

Examples:

Product data:

Laravel/PostgreSQL

NOT:

Laravel/PostgreSQL
+
localStorage
+
hardcoded Astro JSON

Orders:

Laravel/PostgreSQL

NOT:

localStorage
+
WhatsApp
+
Laravel

CMS:

Filament/Laravel

NOT:

Filament
+
Astro Admin

3. RULE: NO DESTRUCTIVE PRODUCTION OPERATIONS

NEVER execute:

```
php artisan migrate:fresh
```

against production.

NEVER use:

```
php artisan db:wipe
```

against production.

NEVER drop production tables as part of a normal deployment.

Production migrations must use:

```
php artisan migrate --force
```

The target architecture explicitly requires persistent PostgreSQL and additive migrations rather than destructive deployment behavior.

4. RULE: NEVER TRUST THE CLIENT

The browser is untrusted.

Never trust frontend-provided:

- price
- discount
- stock
- payment status
- order total
- customer role
- inventory availability

Laravel must recalculate and validate authoritative values.

5. RULE: NEVER COMMIT SECRETS

Never commit:

- `.env`
- `APP_KEY`
- database passwords
- API keys
- payment credentials
- webhook secrets
- SMTP passwords
- cloud storage credentials

`.env.example` may contain placeholders only.

The architecture explicitly requires secrets to exist only in hosting environment configuration and requires rotation because the current key has already been exposed.

6. RULE: DO NOT REWRITE THE ENTIRE PROJECT

Do NOT:

- replace Astro with another frontend framework
- replace Laravel with another backend framework
- replace Filament with another admin framework
- rewrite the entire UI
- create a new unrelated repository
- introduce microservices
- introduce Kubernetes
- create a mobile app
- introduce an AI chatbot
- create a marketplace

unless explicitly requested later.

The objective is controlled migration.

7. RULE: ONE PHASE AT A TIME

Never implement Phase 1 + Phase 2 + Phase 3 simultaneously.

For every phase:

```
Inspect
↓
Plan
↓
Implement
↓
Test
↓
Security check
↓
Build
↓
Verify
↓
Document
↓
Commit
↓
Proceed
```

If a phase fails, stop.

Do not continue into the next phase.

8. REQUIRED DOCUMENTATION

Create or maintain:

```
/docs/

NIES-Platform-Mission-Architecture.md
NIES-Platform-Conversion-Roadmap.md
NIES-Agent-Execution-Rules.md
MIGRATION-LEDGER.md
ARCHITECTURE-DECISIONS.md
DATABASE-MIGRATION.md
API-CONTRACT.md
SECURITY.md
DEPLOYMENT.md
TESTING.md
```

Documentation must be updated in the same change that modifies the architecture.

The architecture itself identifies stale documentation as one of the problems in the current repositories.

9. MIGRATION LEDGER

Create:

```
docs/MIGRATION-LEDGER.md
```

Use this structure:

Current	Target	Action	Status	Notes
LocalStorage CMS	Filament	Remove	Pending	
LocalStorage orders	PostgreSQL orders	Replace	Pending	
Product	Product	Refactor	Pending	
Customer model	Customer + ClientLogo	Split	Pending	
SQLite	PostgreSQL	Migrate	Pending	
Local uploads	R2/S3	Migrate	Pending	
<code>/admin/login</code>	Filament Auth	Delete	Pending	
<code>/api/v1/cms-status</code>	None	Delete	Pending	
Astro middleware	DNS	Delete	Pending	
<code>migrate:fresh</code>	<code>migrate --force</code>	Replace	Pending	

Every migration decision must be recorded here.

10. PHASE 0 — REPOSITORY BASELINE

Objective

Understand exactly what currently exists before touching production architecture.

Tasks

Clone both repositories.

Inspect:

```
frontend_for_Nies  
backend_for_Nies
```

Create a baseline inventory.

Frontend audit

Inspect:

```
package.json  
astro.config.*  
src/  
public/  
src/pages/  
src/components/  
src/layouts/  
src/lib/  
middleware.*
```

Identify:

- routes
 - components
 - layouts
 - API utilities
 - LocalStorage usage
 - hardcoded product data
 - hardcoded settings
 - forms
 - cart logic
 - checkout logic
 - admin functionality
 - deployment configuration
 - GitHub Pages configuration
 - environment variables
-

Backend audit

Inspect:


```
composer.json
.env.example
app/
routes/
database/
config/
resources/
docker/
Dockerfile
docker-entrypoint.sh
```

Identify:

- models
- migrations
- controllers
- services
- observers
- policies
- Filament resources
- authentication
- admin routes
- API routes
- deployment hooks
- database configuration
- storage configuration
- tests

11. BASELINE REPORT

Before modifying code, create:

```
docs/CURRENT-STATE-AUDIT.md
```

It must contain:

```
Frontend architecture
Backend architecture
Database architecture
Authentication
Admin
API
```

Deployment
Storage
Security
Commerce
CMS
Known bugs
Known duplicated functionality
Migration candidates

Do not proceed until the baseline audit exists.

12. PHASE 0 ACCEPTANCE CRITERIA

The phase is complete only when:

- both repositories have been inspected
 - current routes are documented
 - current models are documented
 - current migrations are documented
 - current admin system is documented
 - current deployment process is documented
 - LocalStorage usage is identified
 - security problems are identified
 - duplicate systems are identified
 - migration candidates are recorded
-

13. PHASE 1 — SECURITY EMERGENCY

This phase must happen before feature development.

The architecture identifies exposed credentials, debug configuration, duplicate authentication and a credential-leaking CMS status endpoint as critical issues.

Tasks

1. Rotate APP_KEY

Generate a new production key.

Do not expose it in code.

2. Rotate every exposed credential

Assume every credential committed to Git is compromised.

Rotate:

- database credentials
 - admin credentials
 - API credentials
 - payment credentials
 - webhook secrets
 - SMTP credentials
 - storage credentials
-

3. Remove `.env` from Git

Verify:

```
git ls-files .env
```

must return nothing.

Update `.gitignore`.

4. Remove secrets from Git history

If secrets have previously been committed:

- identify them
- rotate them
- remove them from repository history where appropriate
- verify they are no longer exposed

Do not rely solely on deleting the current file.

5. Disable production debugging

Production:

```
APP_DEBUG=false
```

6. Remove custom admin authentication

Delete:

```
/admin/login
```

if it duplicates Filament authentication.

7. Remove credential-leaking endpoint

Delete:

```
/api/v1/cms-status
```

if it exposes credentials or internal authentication information.

8. Filament becomes the only admin authentication system

Use:

```
manage.niengineeringbd.com
```

for administration.

Enable:

- strong password
- 2FA
- authorization
- session security

The target architecture explicitly specifies Filament's built-in authentication as the only admin authentication system.

14. SECURITY ACCEPTANCE TEST

Verify:

```
/admin/login → removed  
/api/v1/cms-status → removed  
APP_DEBUG → false  
.env → not tracked  
APP_KEY → environment only  
admin → Filament only  
2FA → enabled
```

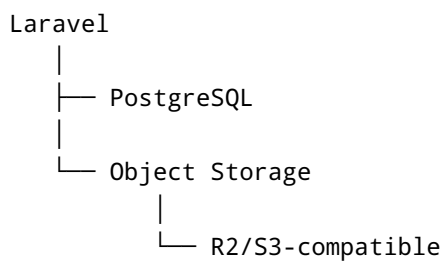
Do not continue until all pass.

15. PHASE 2 — PRODUCTION DATA FOUNDATION

Objective

Create persistent infrastructure before rebuilding commerce.

Target:



The architecture explicitly replaces SQLite-on-ephemeral-disk with managed PostgreSQL and requires persistent media storage.

16. PostgreSQL

Provision managed PostgreSQL.

Configure Laravel.

Verify:

```
php artisan migrate:status
```

17. Object storage

Configure Cloudflare R2 or equivalent S3-compatible storage.

Use it for:

- product images
- project images
- blog images
- logos
- PDFs
- certificates
- future documents

Do not store production media inside ephemeral application storage.

18. DATABASE MIGRATION

Create additive migrations.

Target core schema:

```
categories
products
media

customers
addresses

carts
cart_items

orders
order_items
payments

quote_requests

projects
blog_posts
client_logos
contact_submissions
```

```
site_settings
users
```

The architecture document specifies this core data model, including product pricing/stock, customer/order/payment entities and quote requests.

19. PRODUCT SCHEMA

Ensure product has:

```
id
category_id
sku
title
slug
description
specifications
price
compare_at_price
stock_quantity
track_inventory
status
is_featured
meta_title
meta_description
timestamps
```

The current architecture specifically identifies the absence of price and stock as a core defect.

20. CUSTOMER MODEL

Create a real:

```
Customer
```

Do not confuse customers with client logos.

The current confusing `Customer` content model should become:

ClientLogo

while the actual commerce customer becomes:

Customer

The target architecture explicitly defines this separation.

21. ORDER MODEL

Create:

Order
OrderItem
Payment

Order must contain historical snapshots:

```
customer_name  
phone  
email  
shipping_address  
product_title_snapshot  
unit_price_snapshot  
quantity  
line_total
```

This ensures historical orders remain correct even if product information changes later.

22. CART MODEL

Create:

Cart
CartItem

Support:


```
authenticated customer
```

and:

```
guest session_token
```

Guest cart may be client-assisted.

Final order may never be client-only.

23. DATABASE DEPLOYMENT CHANGE

Replace:

```
php artisan migrate:fresh --seed
```

with:

```
php artisan migrate --force
```

Never use destructive migration commands in production.

24. DATA MIGRATION

Before removing old structures:

1. backup database
2. map old tables to new tables
3. migrate reusable data
4. validate row counts
5. validate relationships
6. verify media references
7. test application
8. only then remove obsolete structures

Never destroy existing business content merely to simplify the migration.

25. PHASE 2 ACCEPTANCE CRITERIA

The phase passes when:

```
PostgreSQL connected
R2 connected
Migrations successful
Existing content preserved
Product schema correct
Customer schema correct
Order schema exists
Payment schema exists
QuoteRequest schema exists
No destructive production migration
```

26. PHASE 3 — ADMIN CATALOG VERTICAL SLICE

Do not build the entire commerce platform yet.

Build one complete product pipeline.

Flow:

```
Filament
↓
Category
↓
Product
↓
Image
↓
Publish
↓
Laravel API
↓
Astro
↓
Customer
```

27. FILAMENT PRODUCT MANAGEMENT

Create/administer:

Category
Product
Media

Product form must support:

- title
- SKU
- slug
- description
- specifications
- price
- compare-at price
- stock
- inventory tracking
- category
- status
- featured
- SEO fields
- images

28. PRODUCT API

Implement:

`GET /api/v1/categories`

`GET /api/v1/products`

`GET /api/v1/products/{slug}`

Support:

- pagination
- category filtering
- search
- featured products
- published status

- stable response schema
-

29. API CONTRACT

Create:

```
docs/API-CONTRACT.md
```

Document:

- endpoint
- method
- authentication
- request body
- response
- validation errors
- HTTP status
- pagination
- examples

Do not allow frontend code to depend on undocumented response structures.

30. ASTRO PRODUCT PIPELINE

Replace hardcoded product data.

Target:

```
Astro
↓
API
↓
Laravel
↓
PostgreSQL
```

Marketing pages remain statically generated where appropriate.

The architecture intentionally keeps Astro static for SEO and speed while allowing client-side commerce operations to call Laravel directly.

31. VERTICAL SLICE TEST

Create one real product:

```
ABC Fire Extinguisher
SKU: ABC-4KG
Price: test value
Stock: test value
```

Verify:

```
Admin creates it
↓
Database contains it
↓
API returns it
↓
Astro displays it
↓
Customer can open product page
```

If any step fails, stop.

32. PHASE 4 — CART

Implement real cart behavior.

Endpoints:

```
GET /api/v1/cart
POST /api/v1/cart/items
PATCH /api/v1/cart/items/{id}
DELETE /api/v1/cart/items/{id}
```

Server must:

- validate product
- validate stock
- calculate authoritative price
- calculate line totals

- reject unavailable products
 - prevent invalid quantities
-

33. CART PRICE RULE

Client may send:

```
{  
  "product_id": 123,  
  "quantity": 2  
}
```

Client must NOT be authoritative for:

```
price  
subtotal  
discount  
total
```

Laravel calculates these.

34. PHASE 5 — COD ORDER VERTICAL SLICE

Do payment gateway later.

First implement:

```
Product  
↓  
Cart  
↓  
Checkout  
↓  
COD  
↓  
Order  
↓  
Filament
```

35. CHECKOUT API

Implement:

```
POST /api/v1/orders
```

Request should contain customer and shipping information.

Laravel must:

1. validate request
2. retrieve products from database
3. verify stock
4. calculate prices
5. calculate subtotal
6. calculate shipping
7. calculate total
8. create customer if required
9. create order
10. create order items
11. create COD payment record
12. reserve/decrement stock according to business rule
13. return order number

36. ORDER STATUS

Use the target order states:

```
pending  
confirmed  
processing  
shipped  
delivered  
cancelled
```

Payment:

```
unpaid  
paid
```

```
failed
refunded
```

The architecture specifies these order and payment states.

37. ORDER HISTORY SNAPSHOT

At order creation store:

```
product title
SKU
price
quantity
customer name
phone
email
shipping address
```

Never reconstruct historical orders solely from current product data.

38. FILAMENT ORDER MANAGEMENT

Admin must be able to:

```
view order
view customer
view items
view total
view payment status
change order status
add internal notes
```

39. COD ACCEPTANCE TEST

Create a real test order.

Expected:


```
Customer
↓
Checkout
↓
POST /api/v1/orders
↓
PostgreSQL
↓
NIES order number
↓
Filament
```

Example:

```
NIES-2026-00001
```

The order must exist independently of WhatsApp.

This is the most important milestone of the entire migration.

40. PHASE 6A — PAYMENT GATEWAY

Only after COD works.

Integrate the chosen Bangladeshi payment gateway.

The architecture specifies SSLCommerz or an equivalent gateway with webhook-based payment state updates.

41. PAYMENT FLOW

Required:

```
Customer
↓
Laravel creates payment
↓
Gateway
↓
```

```
Customer payment
↓
Gateway webhook
↓
Laravel
↓
Verify transaction
↓
Payment updated
↓
Order payment_status updated
```

Never:

```
Frontend says payment successful
↓
Laravel trusts it
```

42. PAYMENT WEBHOOK

Create secure webhook handling.

Must include:

- signature verification where supported
- transaction verification
- idempotency
- duplicate webhook protection
- amount verification
- order verification
- status validation
- logging

A webhook received twice must not create two payments.

43. PHASE 6B — PRODUCTION DNS

Configure:

```
niengineeringbd.com
↓
Astro host

api.niengineeringbd.com
↓
Laravel API

manage.niengineeringbd.com
↓
Filament
```

Do NOT use Astro middleware to fake subdomain routing.

The target architecture explicitly removes the middleware approach because the frontend is static.

44. REMOVE ASTRO MIDDLEWARE SUBDOMAIN LOGIC

After DNS works:

Remove obsolete:

```
middleware.ts
```

subdomain-routing behavior.

Before deletion verify:

```
main site works
API works
admin works
assets work
SEO URLs work
```

45. PHASE 6C — FRONTEND REWIRE

Create:

```
src/lib/api/
```

Recommended:

```
client.ts  
products.ts  
categories.ts  
cart.ts  
checkout.ts  
orders.ts  
quotes.ts  
blog.ts  
projects.ts  
services.ts
```

46. API CLIENT

Centralize:

- API base URL
- request headers
- error handling
- JSON parsing
- authentication/session handling
- timeout behavior
- retry behavior where safe

Do not scatter raw:

```
fetch(...)
```

throughout the entire frontend.

47. REMOVE LOCALSTORAGE BUSINESS DATA

Remove LocalStorage usage for:

```
products  
orders
```

```
CMS content
inventory
prices
payment state
```

Allowed:

```
guest cart identifier
temporary UI state
non-authoritative preferences
```

The target architecture explicitly requires checkout actions to become real API calls instead of browser-local writes.

48. PHASE 7 — CRM + QUOTE REQUESTS

Implement:

```
QuoteRequest
```

API:

```
POST /api/v1/quote-requests
```

Admin:

```
New
Contacted
Quoted
Closed
```

The target architecture separates quote requests from normal orders because large engineering projects are not normal cart purchases.

49. QUOTE FLOW

```
Customer
↓
Service/project page
↓
Request Quote
↓
QuoteRequest
↓
Filament
↓
Staff contacts customer
↓
Quotation
↓
Customer decision
```

50. PHASE 8 — SERVICES

After commerce and quote flow are stable, build:

```
Services
Service Requests
Service Visits
Technicians
Service Records
```

Example services:

```
Fire Extinguisher Refilling
Fire Safety Inspection
Fire Alarm Installation
Fire Hydrant Installation
Fire Pump Installation
CCTV Installation
Maintenance
```

51. SERVICE FLOW

```
Customer
↓
Request Service
↓
Service Request
↓
Admin
↓
Assign technician
↓
Site visit
↓
Findings
↓
Quotation if required
↓
Approval
↓
Service
↓
Completion
```

52. PHASE 9 — FIRE SAFETY ASSET LIFECYCLE

This should become a future differentiator.

Implement:

```
Customer
↓
Extinguisher Asset
↓
Serial Number
↓
Inspection History
↓
Refill History
↓
Service History
```

↓
Next Service Date

Later:

AMC
↓
Visits
↓
Renewal
↓
Billing

Do not implement the entire AMC ERP prematurely.

Start with the data model and service history.

53. PHASE 10 — INVENTORY EVOLUTION

Initially:

`product.stock_quantity`

is acceptable for the first production slice.

When business requirements demand it, evolve to:

Product
↓
Variant
↓
Inventory Location
↓
Inventory Transaction

Support:

purchase
sale
reservation


```
release
return
adjustment
transfer
damage
```

Do not build elaborate warehouse management until actual requirements justify it.

54. PHASE 11 — CONTENT SYSTEM

Migrate/maintain:

```
Projects
Blog
Client Logos
Site Settings
Contact Submissions
```

All managed through Filament.

Public frontend reads from API/build-time API.

No second CMS.

55. PHASE 12 — NOTIFICATIONS

On order:

```
Order created
↓
Customer confirmation
↓
Staff notification
```

Possible channels:

```
Email
SMS
```

Telegram
Slack

Choose only channels actually monitored by staff.

The architecture specifically calls for staff and customer notifications rather than relying on manual WhatsApp messages.

56. PHASE 13 — DEPLOYMENT PIPELINE

Fix:

DeployWebhookService

and GitHub Actions event mismatch.

The architecture notes that the CMS trigger currently dispatches an event type that does not match the workflow listener.

Choose one event name.

For example:

build-static-site

Then make both systems use exactly that value.

57. DEPLOYMENT PIPELINE ACCEPTANCE TEST

Test:

```
Admin publishes content
↓
Deploy trigger
↓
GitHub Action
↓
Astro build
↓
```

```
Deployment
↓
New content visible
```

Do not mark deployment complete merely because the webhook request returned HTTP 200.

Verify the entire chain.

58. PHASE 14 — STAGING ENVIRONMENT

Create:

```
Production
Staging
Local
```

Example:

```
niengineeringbd.com
staging.niengineeringbd.com

api.niengineeringbd.com
api-staging.niengineeringbd.com

manage.niengineeringbd.com
manage-staging.niengineeringbd.com
```

Do not test dangerous migrations directly against production.

59. PHASE 15 — AUTOMATED TESTING

Minimum backend tests:

```
Product API
Category API
Cart
Cart validation
Checkout
Order creation
```

```
Order totals
Stock validation
COD
Payment webhook
Quote request
Authentication
Authorization
```

60. COMMERCE TEST CASES

Test:

```
normal product
out-of-stock product
zero quantity
negative quantity
invalid product
changed price
changed stock
duplicate checkout
duplicate webhook
cancelled order
guest checkout
customer checkout
```

61. SECURITY TESTING

Run:

```
dependency scanning
SAST
secret scanning
authorization tests
authentication tests
rate-limit tests
file-upload tests
API validation tests
payment webhook tests
```

Never expose:

```
database credentials
APP_KEY
payment secrets
admin passwords
internal stack traces
```

62. FRONTEND TESTING

Verify:

```
Home
Products
Product detail
Category
Blog
Projects
Contact
Quote request
Cart
Checkout
Order confirmation
```

Test:

```
mobile
tablet
desktop
slow network
API failure
empty data
out-of-stock
invalid checkout
```

63. SEO TESTING

Verify:

```
title
meta description
canonical
Open Graph
robots
sitemap
structured data where appropriate
404
redirects
clean slugs
```

64. PERFORMANCE TESTING

Check:

```
homepage load
product page load
image optimization
JavaScript size
API latency
database query count
build time
```

Do not sacrifice Astro's static/SEO strengths unnecessarily.

65. MEDIA TESTING

Verify:

```
upload
storage
retrieve
delete
replace
alt text
image dimensions
invalid MIME
```

```
oversized file  
broken URL
```

Verify that redeploying the application does NOT delete uploaded media.

66. BACKUP TEST

Backups are not complete until restoration is tested.

Perform:

```
backup  
↓  
create test restore  
↓  
restore PostgreSQL  
↓  
verify records  
↓  
verify media  
↓  
verify application
```

Document the procedure in:

```
docs/DEPLOYMENT.md
```

67. OBSERVABILITY

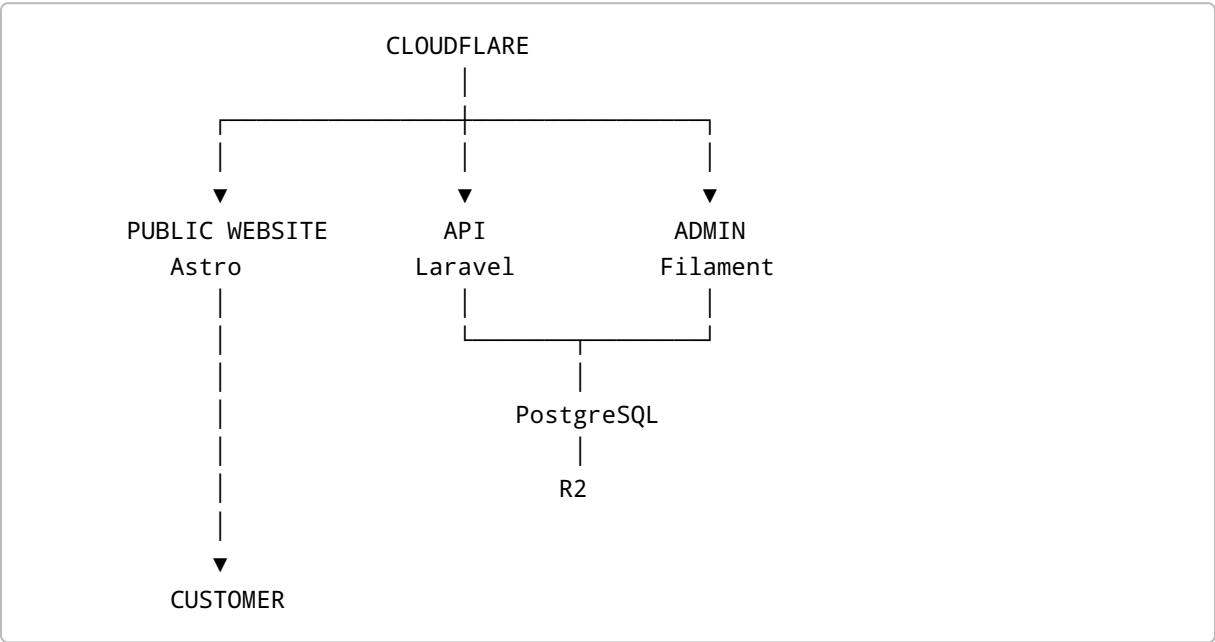
Add:

```
application error monitoring  
database monitoring  
deployment logs  
payment logs  
webhook logs  
important admin audit logs
```

An error monitor such as Sentry may be used if appropriate; the architecture identifies this as part of hardening.

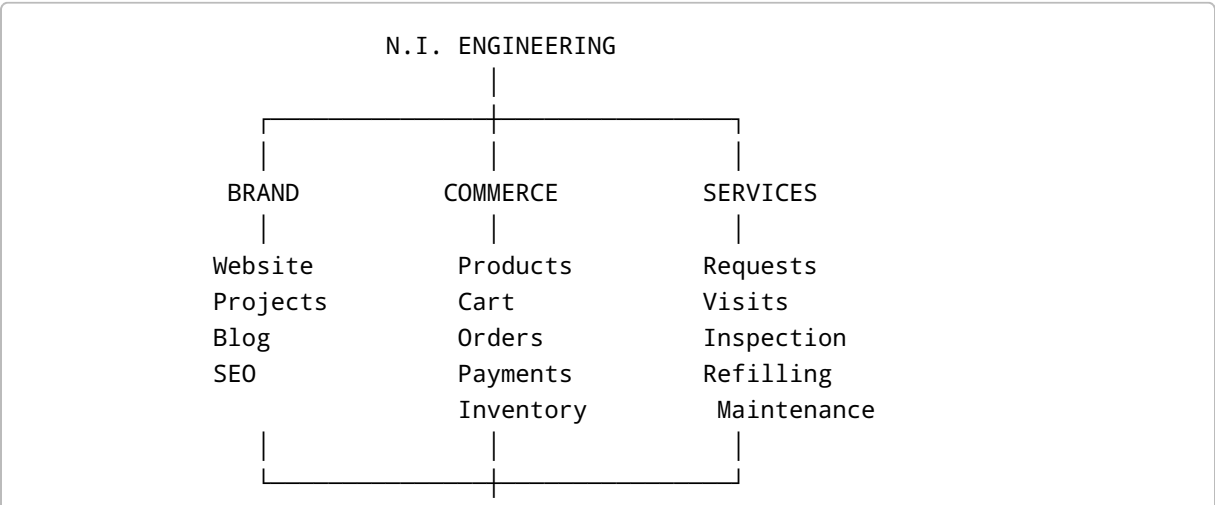
68. FINAL ARCHITECTURE

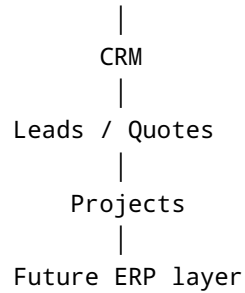
The completed system should look like:



The public website remains static and SEO-oriented, while Laravel owns dynamic business operations.

69. FINAL BUSINESS ARCHITECTURE





70. DEFINITION OF PRODUCTION READY

Do NOT declare the project complete because:

- the homepage looks good
- the build succeeds
- the admin opens
- a product page renders

Production V1 is complete only when:

```
Admin can create product
↓
Customer can see product
↓
Customer can add product to cart
↓
Customer can checkout
↓
Laravel creates real order
↓
Order persists in PostgreSQL
↓
Order appears in Filament
↓
COD or verified online payment works
↓
Staff can process order
```

AND:

```
Customer can request quote
↓
```

Staff receives request



Staff can manage it

AND:

Customer can request service



Staff can manage request

AND:

Existing content survives deployment

AND:

Secrets are not exposed

AND:

Production deployments never destroy data

71. AGENT STOP CONDITIONS

The agent **MUST** stop and report instead of guessing when:

1. Existing data structure is ambiguous.
2. A migration may destroy data.
3. A payment integration behavior is unclear.
4. Credentials are required but unavailable.
5. Production infrastructure is unavailable.
6. A business rule is undefined.
7. Two existing systems conflict.
8. The target architecture does not define the required behavior.
9. A requested change would require breaking an existing production feature.
10. Security cannot be verified.

Do not invent business rules.

Do not silently choose destructive behavior.

72. AGENT REPORT FORMAT

At the end of every phase, report:

Phase

Phase X – Name

Completed

- ...
- ...
- ...

Files changed

```
path/to/file
path/to/file
```

Database changes

```
migration...
table...
column...
```

API changes

```
GET /api/v1/...
POST /api/v1/...
```

Tests

```
PASS
PASS
FAIL
```

Security

PASS / FAIL

Known issues

- ...

Decisions

- ...

Next phase

Phase X+1

Do not proceed automatically if a blocking issue exists.

73. GIT COMMIT RULE

Use small, meaningful commits.

Examples:

```
security: remove exposed admin authentication
security: remove cms-status endpoint
security: disable production debug
database: add product pricing and inventory
database: create customer model
database: create order domain
api: add product endpoints
api: add cart endpoints
commerce: implement cod checkout
admin: add order management
frontend: replace product local storage with api
deployment: fix webhook event mismatch
```

Never create one enormous:

"fixed everything"

commit.

74. BRANCH STRATEGY

Use:

```
main
staging
migration
```

During migration:

```
migration
  ↓
phase branch
  ↓
tests
  ↓
staging
  ↓
verification
  ↓
main
```

Production must never receive untested AI-generated changes directly.

75. AI AGENT PRIORITY ORDER

When requirements conflict, use this priority:

1. Security
2. Data integrity
3. Business correctness
4. API correctness
5. Testability
6. Maintainability
7. Performance
8. SEO

- 9. UX
- 10. Visual polish

A beautiful UI that creates incorrect orders is a failure.

A slower but correct checkout is preferable to a fast broken checkout.

76. WHAT THE AGENT MUST NOT DO

Never:

```
invent a second database
invent a second CMS
invent a second authentication system
invent a second order system
store authoritative orders in localStorage
trust frontend prices
trust frontend payment status
use destructive production migrations
commit secrets
hardcode production credentials
rewrite the framework without approval
introduce unnecessary microservices
skip tests because the code "looks correct"
```

77. FIRST COMMAND / FIRST ACTION

Do NOT immediately start coding.

The first action should be:

```
AUDIT ONLY
```

The agent must inspect both repositories and produce:

```
CURRENT-STATE-AUDIT.md
MIGRATION-LEDGER.md
```

No production modifications.

No destructive changes.

No feature development.

No framework replacement.

78. FIRST EXECUTION PROMPT

Use the following instruction as the first message to the AI coding agent:

You are migrating the existing N.I. Engineering Services platform.

Repositories:

Frontend: https://github.com/Naz365/frontend_for_Nies

Backend: https://github.com/Naz365/backend_for_Nies

You have two governing documents:

1. NIES-Platform-Mission-Architecture.md
2. NIES-Platform-Conversion-Roadmap.md

Treat those documents as the architectural contract.

Your first task is NOT to code.

Perform a read-only audit of both repositories.

Inspect:

- frontend architecture
- backend architecture
- database
- migrations
- models
- controllers
- services
- API
- authentication
- Filament
- LocalStorage
- checkout

- product system
- deployment
- Docker
- GitHub Actions
- environment configuration
- media/storage
- security
- tests

Then create:

```
docs/CURRENT-STATE-AUDIT.md
```

and:

```
docs/MIGRATION-LEDGER.md
```

Classify every important existing component as:

KEEP REFACTOR REPLACE DELETE UNKNOWN

For every DELETE or REPLACE decision, explain:

1. current behavior
2. problem
3. target replacement
4. migration risk
5. data-preservation requirement

Do not modify production behavior during this audit.

Do not run destructive database commands.

Do not create new architecture.

Do not introduce new frameworks.

Do not commit secrets.

After completing the audit, STOP.

Report the findings and wait for approval to execute Phase 1.

79. SECOND EXECUTION PROMPT — PHASE 1

After the audit has been reviewed, give the agent:

Execute Phase 1 — Security Emergency only.

Do not implement Phase 2 or later.

Perform:

1. credential rotation preparation
2. remove tracked `.env`
3. ensure `.env.example` contains placeholders only
4. set production debug configuration correctly
5. remove duplicate `/admin/login`
6. remove `/api/v1/cms-status`
7. configure Filament authentication as the only admin authentication
8. configure strong admin credentials
9. enable 2FA
10. run security checks

Before deleting anything, verify dependencies.

After implementation:

- run tests
- run build
- inspect routes
- inspect authentication
- verify no secrets remain in tracked files
- update `MIGRATION-LEDGER.md`
- update `SECURITY.md`

Do not touch commerce, database migration, payment, or frontend architecture.

STOP after Phase 1.

80. SUBSEQUENT EXECUTION METHOD

For every later phase, the prompt must follow:

EXECUTE PHASE X ONLY

Read:

- Mission Architecture
- Conversion Roadmap
- Agent Execution Rules
- Current State Audit
- Migration Ledger

Before coding:

- inspect affected files
- identify dependencies
- identify data risks

Implement only Phase X.

Required tests:

...

Required acceptance criteria:

...

Update:

- Migration Ledger
- API Contract if required
- Database documentation if required
- Architecture Decisions if required

Run:

- tests
- build
- lint/static analysis
- security checks where relevant

Report:

- completed
- files changed
- migrations
- API changes
- tests
- security
- known issues

STOP.

81. ULTIMATE AGENT OBJECTIVE

The final objective is NOT:

"Make the current website work."

The objective is:

Convert N.I. Engineering Services from a frontend-centric website with a partially implemented Laravel backend into one coherent, persistent, secure, API-driven engineering commerce platform where the database is the source of truth, Filament is the single administration system, Astro is the public presentation layer, and real orders/services/quotes become durable business records.

The target architecture's mission is explicitly centered on reliable products, real stock/pricing, genuine checkout/order persistence, a single secure admin system, and documentation that accurately represents the deployed system.

FINAL SUCCESS CONDITION

The migration is successful when a new developer can clone the repositories six months later and understand:

```
Where data lives
    ↓
PostgreSQL

Where files live
    ↓
R2 / object storage

Where business logic lives
    ↓
Laravel

Where admin lives
    ↓
Filament

Where public UI lives
    ↓
Astro
```

Where APIs live



/api/v1

Where production website lives



niengineeringbd.com

Where API lives



api.niengineeringbd.com

Where admin lives



manage.niengineeringbd.com

There must be no hidden second CMS, no browser-based order database, no destructive deployment process, and no undocumented parallel architecture.